

Práctica 1 - IA Generativa

Luca Bellu e Iván Domínguez

Problema 1

Problema de las N reinas.

- Evalúa cuantas reinas caben (atacan todas las posiciones del tablero) en un tablero de ajedrez $N \times N$. Naturalmente, tiene en cuenta que un par de reinas no pueden compartir fila, columna ni diagonal.
- Sabiendo que es un problema con soluciones para cualquier N , emplearemos un SAT-Caminar por cuestiones de eficiencia.
- Referencia: Rostami, T., & Bright, C. (2026). Queen domination by SAT solving. arXiv. <https://arxiv.org/abs/2508.11945>.

Planteamiento del problema

Restricciones

Llamaremos $x_{i,j}$ a la cláusula que es verdadera cuando hay una reina en i,j y negativa en caso contrario.

- Al menos una reina por fila i , como máximo una reina por fila i .

$$x_{i,j1} \vee x_{i,j2} \vee \cdots \vee x_{i,jN} \equiv \bigvee_{j=1}^N x_{i,j} , \neg(x_{i,j1} \wedge x_{i,j2} \wedge \cdots \wedge x_{i,jN}) \equiv \bigvee_{j=1}^N \neg x_{i,j}$$

- Al menos una reina por columna j , como máximo una reina por columna j .

$$x_{i1,j} \vee x_{i2,j} \vee \cdots \vee x_{iN,j} \equiv \bigvee_{i=1}^N x_{i,j} , \neg(x_{i1,j} \wedge x_{i2,j} \wedge \cdots \wedge x_{iN,j}) \equiv \bigvee_{i=1}^N \neg x_{i,j}$$

Planteamiento del problema

Más restricciones

- Como máximo una reina por diagonal principal. Todas las celdas comparten la misma diferencia de coordenadas $i_1 - j_1 = i_2 - j_2 = \dots = i_N - j_N$.
- Como máximo una reina por diagonal secundaria. Todas las celdas comparten misma suma de coordenadas $i_1 + j_1 = i_2 + j_2 = \dots = i_N + j_N$.
- La restricción única será entonces $|i_k - i_{k+1}| = |j_k - j_{k+1}|$ y entonces

$$\bigvee_{k=1}^N \neg x_{ik,jk}$$

SAT Solver: SAT-Caminar

Imaginemos la $BC = \{C_1 \equiv A \vee B, C_2 \equiv \neg A \vee C, C_3 \equiv \neg B \vee \neg C\}$

- Paso 1: asignación inicial aleatoria. Por ejemplo $A = F, B = F, C = F$
- Paso 2: evaluación del modelo. C_1 no se cumpliría.
- Paso 3: selección e intercambio. Con probabilidad p arbitraria, elige entre:
 - Estrategia de pasada-aleatoria: invierte A o B aleatoriamente.
 - Estrategia de min-conflictos: evalúa qué cambio funcionará sin romper la satisfacibilidad de otras cláusulas. Cambiar A rompe otras, B no.

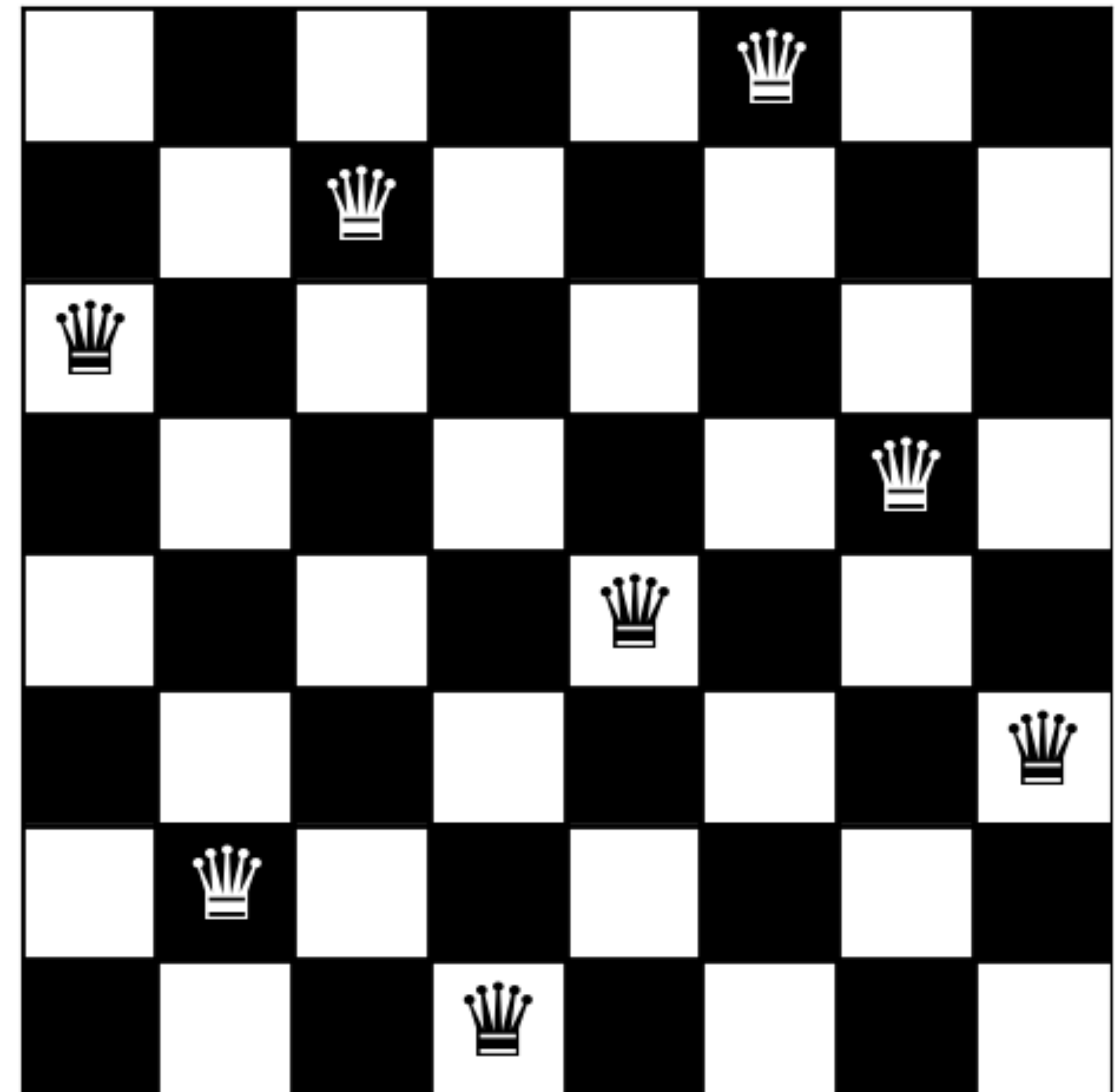
Se han usado las restricciones ya
optimizadas del paper.

Resultados (1)

N=8

Tablero convencional

Solución SAT-CAMINAR para un tablero 8x8

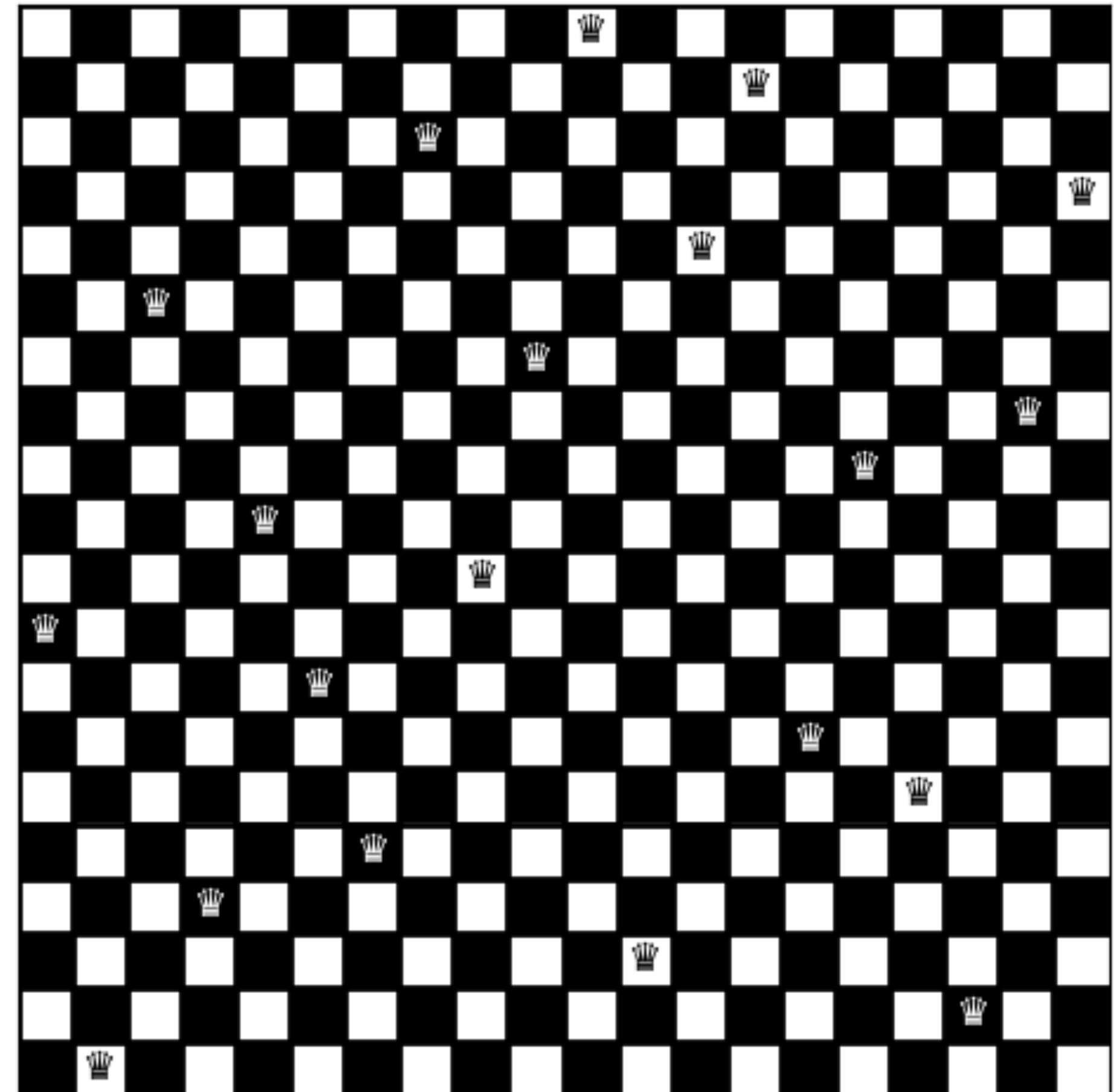


Resultados (2)

N=20

Unos 3 segundos.

Solución SAT-CAMINAR para un tablero 20x20



Ejercicio 2

Mismo problema, más ambicioso

- Usando lógica de primer orden podemos atacar un problema parecido pero más ambicioso.
- Si ya no se exige la restricción de no amenazarse entre sí, el problema pasa a ser de optimización (encontrar el menor número de reinas).
- Más nativo en lógica de predicados, en lugar de generar cientos de cláusulas independientes dinámicamente, el universo queda acotado mediante axiomas universales gracias a las facilidades que brinda la lógica de primer orden.

Razones

- Escalabilidad: la FNC es menos nativa y depende del tamaño del tablero, el número de cláusulas es $O(N^2)$. Con lógica de primer orden esto no ocurre, los axiomas ocupan una línea.
- No hay restricciones cruzadas en cada casilla del tablero, con los cuantificadores $\forall$ y $\exists$ se codifican reglas universales. Es fácil ver que $\bigvee_{n=i}^k \equiv \exists$ y $\bigwedge_{n=i}^k \equiv \forall$.
- Ya no hay que pre-calcular las posiciones de las diagonales.

Formalización

- Predicados: $\text{Reina}(x, y)$ $\text{Ataque}(x_1, y_1, x_2, y_2)$
- Definición geométrica del ataque:
 - Compartición de filas, columnas o diagonales es definido ataque.
 - $\forall x_1 \forall y_1 \forall x_2 \forall y_2 (\text{Ataque}(x_1, y_1, x_2, y_2) \iff (x_1 = x_2 \vee y_1 = y_2 \vee |x_1 - x_2| = |y_1 - y_2|))$
- Restricción de dominación:
 - Toda casilla está siendo atacada por alguna reina.
 - $\forall x \forall y \exists i \exists j (\text{Reina}(i, j) \wedge \text{Ataque}(i, j, x, y))$
- Restricción de cardinalidad:
 - Para el problema del tablero 5×5 , hay 3 “asientos” disponibles: $(r_1, c_1), (r_2, c_2), (r_3, c_3)$
 - $\exists r_1 \exists c_1 \exists r_2 \exists c_2 \exists r_3 \exists c_3 \forall x \forall y (\text{Reina}(x, y) \implies ((x = r_1 \wedge y = c_1) \vee (x = r_2 \wedge y = c_2) \vee (x = r_3 \wedge y = c_3)))$

Desventajas

- Tiempo: aunque lógicamente sea más fácil de entender, la lógica de primer tiene mayores exigencias computacionales, pues tiene que desenrollar las expresiones en lugar de tomar listas de ceros y unos. Además, recordemos que el problema trata de buscar un mínimo, no solo de comprobar la satisfabilidad de la solución.
- Exige comprobar problemas satisfactibles. Si pruebas 2 reinas en un tablero 5×5 , devuelve “no satisfactible”. La solución es correcta, pero solo muestra el tablero cuando el problema es satisfactible.

Microsofts *python z3-solver*

- Framework de python que implementa un motor SMT (Satisfiability Modulo Theories).
- Integra teorías matemáticas de fondo; no resuelve por fuerza bruta.
- Eso le permite integrar de forma nativa los cuantificadores $\forall$ y $\exists$.
- Tambien reconoce estructuras geométricas, por lo que las diagonales no son pre-calculadas, son directas.

Resultados

Solución LPO (Z3): 3 reinas (tablero 5x5)

